

Ontwerp en realisatie van een performante provisioner voor een virtuele Windows-en Linux omgevingen in Rust.

Jens Gosseye.

Scriptie voorgedragen tot het bekomen van de graad van
Professionele bachelor in de toegepaste informatica

Promotor: Dhr. B. Van Vreckem

Co-promotor: Dhr. J. Courtens

Academiejaar: 2025–2026

Eerste examenperiode

Departement IT en Digitale Innovatie .

**HO
GENT**

Woord vooraf

Binnen de opleiding Toegepaste Informatica aan HOGENT wordt in de laatste 2 jaren gewerkt met geautomatiseerde omgevingen om virtuele Windows- en Linux-systemen klaar te zetten. Hierbij wordt gebruikgemaakt van Vagrant, een tool die het mogelijk maakt om labo- en ontwikkelomgevingen op een declaratieve manier aan te maken en provisionen. In de praktijk blijkt echter dat dit proces niet altijd even vlot verloopt. Opstarttijden kunnen traag zijn, provisioning kan vastlopen en bij onderbrekingen of foutmeldingen blijft het proces soms op de achtergrond doorlopen. Dit zorgt voor frustratie en tijdverlies bij studenten en docenten.

Daarom heb ik voor deze bachelorproef gekozen om een eigen provisioner te ontwikkelen in Rust. De keuze voor dit onderwerp sluit aan bij mijn interesse in automatisering en programmeren, ook al volgt deze bachelorproef niet rechtstreeks uit mijn gekozen afstudeerrichting. Een provisioner combineert toch programmeren met systeembeheer en infrastructuur, waardoor het onderwerp goed aansluit bij de bredere richting van toegepaste informatica. Bovendien leek het interessant om na te gaan of een eigen oplossing performanter en gebruiksvriendelijker kan zijn dan de bestaande aanpak met Vagrant.

Samenvatting

Deze bachelorproef behandelt het ontwerp en de realisatie van RustProv, een provisioner voor virtuele Windows- en Linuxomgevingen. Binnen de opleiding Toegepaste Informatica aan HOGENT wordt voor het opzetten van labo- en ontwikkelomgevingen vaak gebruikgemaakt van Vagrant, maar in de praktijk leidt dit soms tot trage opstarttijden, foutgevoelige provisioning en beperkte controle over het proces. Vanuit dit punt werd onderzocht hoe een eigen provisioner in Rust ontwikkeld kan worden die beter aansluit bij de behoeften van een onderwijsomgeving. Het doel van deze bachelorproef is een proof-of-concept te realiseren dat virtuele machines automatisch kan aanmaken, configureren en provisionen op een consistente en betrouwbare manier. Daarbij wordt ook gekeken naar praktische verbeteringen, zoals het direct koppelen van een VDI-bestand, het gebruik van shared folders en het ondersteunen van zowel één VM als meerdere VM's. De performantie van RustProv wordt bovendien vergeleken met die van Vagrant om na te gaan of de nieuwe aanpak sneller of efficiënter is. Deze bachelorproef levert daarmee een werkende basis op voor verdere uitbreiding van geautomatiseerde VM-provisioning in een onderwijscontext.

Inhoudsopgave

Lijst van figuren	vii
Lijst van tabellen	viii
Lijst van codefragmenten	ix
1 Inleiding	1
1.1 Probleemstelling	1
1.2 Onderzoeksvraag	1
1.3 Onderzoeksdoelstelling	2
1.4 Opzet van deze bachelorproef	2
2 Stand van zaken	3
2.0.1 RustProv	3
2.0.2 Virtualisatie en provisioning	3
2.0.3 Waarom een provisioner?	3
2.0.4 Infrastructure as Code	3
2.0.5 Educatieve context	4
2.0.6 Rust als programmeertaal	4
2.0.7 Afbakening van de doelomgeving	4
3 Provisioning	6
3.0.1 Tijdswinsten tijdens het provisioningsproces	6
3.0.2 Provisioningworkflow	7
4 Uitbreidingen	9
4.0.1 Snapshotting	9
4.0.2 Priority Queue	9
4.0.3 Kill	9
4.0.4 Ondersteuning	9
4.0.5 Dependencies	11
5 Benchmark	12
5.0.1 Snelheidsvergelijking met Vagrant	12
5.1 Requirements-analyse	13
5.2 Minimale vereisten	13
5.3 Vereisten voor RustProv	15
5.4 Functionele en niet-functionele vereisten	16

5.5	Afbakening	16
-----	----------------------	----

Lijst van figuren

Lijst van tabellen

5.1	Vergelijking van de Windows-tijden tussen Vagrant en RustProv in seconden.	13
5.2	Vergelijking van de Linux-tijden tussen Vagrant en RustProv in seconden.	13

Lijst van codefragmenten

3.1	Provisioning in Linux	8
3.2	Provisioning in Windows	8
4.1	Zoeken naar de gebruikersmap per besturingssysteem	10
4.2	Zoeken naar het os type	10

1

Inleiding

Binnen de richting IT Infrastructure Engineer aan HOGENT wordt gebruikgemaakt van verschillende provisioningtools om virtuele Windows- en Linuxomgevingen op te zetten. Een veelgebruikte tool is Vagrant: studenten gebruiken deze om ontwikkel- en labo-omgevingen te automatiseren op basis van een declaratieve configuratiefile. In de praktijk treden echter verschillende problemen op tijdens dit proces: virtuele machines starten traag op, het provisioningproces loopt soms vast of wordt niet gestart door time-outs, virtuele machines kunnen onverwachte *kernel panics* vertonen en bij het onderbreken van het provisioningproces blijft dit soms op de achtergrond verder draaien, waardoor de gebruiker het proces manueel moet stopzetten. Dit zorgt voor veel frustraties en tijdverlies, zowel in onderwijs- als bedrijfscontext.

1.1. Probleemstelling

De huidige Vagrant-gebaseerde aanpak voor provisioning in de opleiding Toegepaste Informatica aan HOGENT leidt regelmatig tot trage opstarttijden, vastlopende provisioning en instabiele virtuele machines. Dit zorgt ervoor dat studenten en lectoren tijd verliezen tijdens de lessen, labo's en evaluaties. Hierdoor wordt de focus van de lessen verlegd van het leren naar het oplossen van verschillende technische problemen. De doelgroep van dit onderzoek bestaat uit studenten en docenten binnen de richting IT Infrastructure Engineer aan HOGENT. Zij hebben nood aan een meer performante en betrouwbare provisioningoplossing die beter is afgestemd op een onderwijs- en laboomgeving.

1.2. Onderzoeksvraag

De centrale onderzoeksvraag van deze bachelorproef luidt:

Hoe ontwerp en realiseer je een performante provisioner voor virtuele Windows- en Linuxomgevingen, die beter inspeelt op de behoeften van onderwijs en bedrijven dan de huidige Vagrant-gebaseerde aanpak?

Om deze hoofdonderzoeksvraag te concretiseren, worden volgende deelvragen onderzocht:

- Welke functionaliteiten zijn essentieel voor een provisioner die in onderwijs- en labo-omgevingen wordt gebruikt?
- Op welke besturingssystemen zal de provisioner zelf draaien?
- Voor welke gast-besturingssystemen moet de provisioner virtuele machines kunnen opbouwen en configureren?

1.3. Onderzoeksdoelstelling

Het doel van deze bachelorproef is een proof-of-concept provisioner te schrijven in Rust. Deze zal virtuele Linux- en Windows-omgevingen kunnen opzetten op een betrouwbare manier. De succescriteria omvatten een werkende command-line tool die op Windows kan worden uitgevoerd, en ondersteuning voor kernfunctionaliteiten zoals het opstarten van de VM, provisioning via SSH en configuratie via een declaratief bestand. Daarnaast wordt een aantoonbare verbetering in de provisioningtijd en stabiliteit tegenover de bestaande Vagrant-omgevingen verwacht. Tot slot zal er documentatie en een gebruikershandleiding voorzien worden, zodat de tool inzetbaar is binnen de opleiding Toegepaste Informatica aan HOGENT.

1.4. Opzet van deze bachelorproef

De rest van deze bachelorproef is als volgt opgebouwd:

In Hoofdstuk 2 wordt een overzicht gegeven van de stand van zaken binnen het onderzoeksdomein, op basis van een literatuurstudie.

In Hoofdstuk ?? wordt de methodologie toegelicht en worden de gebruikte onderzoekstechnieken besproken om een antwoord te kunnen formuleren op de onderzoeksvragen.

In Hoofdstuk ??, tenslotte, wordt de conclusie gegeven en een antwoord geformuleerd op de onderzoeksvragen. Daarbij wordt ook een aanzet gegeven voor toekomstig onderzoek binnen dit domein.

2

Stand van zaken

2.0.1. RustProv

RustProv is de naam van de provisioner die in deze bachelorproef wordt ontwikkeld. De naam combineert *Rust*, de gekozen programmeertaal, en *Prov*, een afkorting van provisioning.

2.0.2. Virtualisatie en provisioning

Virtuele machines vormen de basis van de omgevingen waarvoor de provisioner wordt ontwikkeld. In deze bachelorproef ligt de focus op het automatisch aanmaken, configureren en beschikbaar maken van een VM in een onderwijscontext. Virtualisatie laat toe om meerdere geïsoleerde omgevingen op dezelfde fysieke machine te gebruiken, wat handig is voor labo's, testen en demo-omgevingen.

2.0.3. Waarom een provisioner?

Een provisioner automatiseert het opzetten en configureren van virtuele omgevingen. Daardoor verlopen testen en deployments consistent, sneller en met minder kans op fouten. In deze bachelorproef is een provisioner vooral nuttig om een stabiele en herhaalbare omgeving te voorzien voor onderwijs- en labo-opdrachten. Dit vermijdt ook dat dezelfde configuratie telkens manueel opnieuw moet worden uitgevoerd.

2.0.4. Infrastructure as Code

Het automatiseringsproces in systeembeheer wordt vaak aangeduid als *Infrastructure as Code (IaC)*. Binnen IaC wordt een onderscheid gemaakt tussen verschillende soorten tools, zoals *provisioning tools*, *configuration management* en *orchestration*. Provisioningtools richten zich op het creëren van de infrastructuur, terwijl configuratiemanagementtools instaan voor de installatie en configuratie van

software op de server. Het voordeel hiervan is dat de opbouw van een omgeving reproduceerbaar en beter beheersbaar wordt.

Voor lokale ontwikkelomgevingen worden vaak tools zoals Vagrant gebruikt. Daarnaast bestaan er tools zoals Ansible en Terraform, die eerder geassocieerd worden met cloudomgevingen of grotere configuraties. Welke tool gekozen wordt, hangt af van de context, de grootte van de omgeving en de mate van automatisering die nodig is.

2.0.5. Educatieve context

Hoewel er verschillende commerciële oplossingen bestaan voor enterprise-omgevingen, gelden in een onderwijscontext andere eisen voor virtualisatie. Onderzoek naar het gebruik van virtuele machines in het onderwijs toont dat een evenwicht nodig is tussen prestatie, gebruiksvriendelijkheid voor studenten en de beperkingen van consumentenhardware zoals laptops. In een labo-omgeving moet een virtuele machine dus niet alleen werken, maar ook snel beschikbaar zijn.

Ook speelt tijd een grote rol. De *provisioning time*, de tijd tussen het aanvragen van de virtuele machine en de beschikbaarheid ervan, is een kritieke performance-indicator die de productiviteit beïnvloedt. Lange provisioning times kunnen tijdens labo's of examens leiden tot minder effectieve werktijd. Daardoor blijft de vraag bestaan hoe een lichtgewicht en snelle provisioner gerealiseerd kan worden die specifiek afgestemd is op Windows- en Linuxomgevingen in een educatieve context. Dit is extra relevant wanneer meerdere studenten tegelijk met dezelfde tool moeten werken.

2.0.6. Rust als programmeertaal

Rust werd gekozen als programmeertaal voor de provisioner vanwege de goede cross-platform compatibiliteit, waardoor de tool zowel op Linux als op Windows kan draaien zonder grote aanpassingen. De taal combineert prestatie met geheugenveiligheid via het ownership-model, waardoor veelvoorkomende fouten zoals geheugenlekken worden vermeden. Dit is belangrijk voor infrastructuurtools, omdat fouten zoals een onderbroken provisioning of een mislukte SSH-verbinding gecontroleerd moeten kunnen worden afgehandeld. Rust is relatief jong, maar wordt steeds vaker gebruikt voor systeemsoftware en tools waar betrouwbaarheid belangrijk is.

2.0.7. Afbakening van de doelomgeving

De provisioner wordt ontworpen voor de concrete SEP-opdracht en moet de stappen ondersteunen die nodig zijn om die opdracht in een vooraf opgezette virtuele omgeving uit te voeren. Dit maakt de tool doelgericht en beperkt de scope tot wat effectief nodig is voor de opleiding. Door die afbakening blijft het project haalbaar binnen de beschikbare tijd en blijft de focus liggen op een werkende proof-of-

concept. Dit project gebruikt een bridge adapter voor het netwerk en drie test-VM's die eerder voor deze opdracht werden ingezet.

3

Provisioning

3.0.1. Tijdswinsten tijdens het provisioningsproces

VDI-gebaseerde opstart

In plaats van een standaard VirtualBox-image wordt de virtuele schijf rechtstreeks gekoppeld via een VDI-bestand. Dit kan de opstart en provisioning vereenvoudigen doordat er minder extra conversie- of opbouwstappen nodig zijn. Het importeren van een volledige VM kan lang duren, terwijl het koppelen van een bestaande VDI dit proces mogelijk versnelt. Daardoor wordt de omgeving sneller beschikbaar voor gebruik.

Een belangrijk ontwerpdoel is het verkorten van de provisioningtijd. Door een VDI rechtstreeks te koppelen in plaats van met een volledig opgebouwde VirtualBox-image te werken, kan de opstartprocedure mogelijk sneller verlopen. Dit moet later bevestigd worden door een vergelijking tussen verschillende configuraties. De snelheid van dit proces is belangrijk omdat de gebruiker zo minder lang hoeft te wachten vooraleer de omgeving klaar is.

VM's eerst opstarten

Bij het gebruik van meerdere virtuele machines worden eerst alle VM's opgestart voordat de scripts worden uitgevoerd. Hierdoor kan de totale wachttijd verminderd worden, omdat de opstarttijd van de virtuele machine en de SSH-beschikbaarheid niet per machine apart moet worden afgewacht tijdens de provisioning.

Netwerk- en opslagopzet

Naast de VDI-keuze speelt ook de netwerkconfiguratie een rol in de totale opstarttijd. Door de VM meteen via een bridge adapter of vergelijkbare netwerkopzet te verbinden, kan het systeem sneller bereikbaar worden voor verdere provisioning. Dit vermindert de tijd tussen het opstarten van de VM en het moment waarop de provisioner effectief kan beginnen.

Preconfiguratie van boxes

Tijdens het opstarten van de virtuele machines wordt na het aanmaken en starten geen extra configuratie uitgevoerd. Bij Vagrant worden daarentegen nog verschillende eigenschappen van de nieuwe VM aangepast, zoals de hostname en de SSH-sleutel. Deze bijkomende stappen verlengen het opstartproces.

3.0.2. Provisioningworkflow

De provisioner doorloopt een vaste reeks stappen: de VM wordt opgestart, het IP-adres wordt opgehaald, er wordt verbinding gemaakt via SSH en vervolgens worden de provisioningstaken uitgevoerd. Deze reeks gebeurt in een vaste volgorde omdat elke stap afhankelijk is van de vorige. Zonder een correct opgestarte VM kan bijvoorbeeld geen netwerkverbinding worden gemaakt, en zonder SSH kan geen verdere configuratie gebeuren.

1. Aanmaken VM

In deze stap wordt de virtuele machine voorbereid en geregistreerd in de virtualisatieomgeving. Dit vormt de basis van het volledige provisioningproces. Daarna kunnen de netwerk- en opslaginstellingen worden aangepast zodat de machine correct kan starten.

2. Kopiëren van de VDI en aanpassing van de SID

Hier wordt de virtuele schijf gekopieerd of hergebruikt en worden de nodige instellingen aangepast zodat de VM correct kan opstarten. Deze stap zorgt ervoor dat de nieuwe omgeving een eigen identiteit krijgt en niet conflicteert met andere virtuele machines. Ook kunnen hier eventueel machine-specifieke instellingen worden ingesteld.

3. Bridgeadapter

Er wordt gebruikgemaakt van een bridgeadapter, zodat een tweede netwerk-interface aan de VM wordt toegevoegd. Deze keuze werd gemaakt omwille van de eenvoud. Het toevoegen van extra netwerkadapters zou de opzet complexer maken en extra elementen introduceren die voor deze benchmarks niet nodig zijn.

4. Shared folders

In deze fase wordt de `./script`-map gemount aan de virtuele machine. Hierdoor worden bestanden en scripts van de host direct toegankelijk binnen de VM. De shared folder wordt gebruikt zodat de provisioner, eenmaal verbonden via SSH, de benodigde scripts kan uitvoeren die zich in deze map bevinden.

5. NAT Port Forwarding

De netwerkconfiguratie wordt aangepast zodat de host toegang krijgt tot de virtuele machine via port forwarding. Dit is nodig om later via SSH verbinding

te kunnen maken. Zonder deze stap zou de provisioner geen betrouwbare communicatie met de VM kunnen opzetten.

6. Opstarten van de VM

Na de configuratie wordt de virtuele machine effectief opgestart. Op dit moment wordt gecontroleerd of de VM correct reageert en of de benodigde netwerkparameters beschikbaar zijn. Deze stap is essentieel voordat de provisioning verder kan gaan.

7. SSH en shared folder

Zodra de VM beschikbaar is, wordt via SSH verbinding gemaakt en kunnen gedeelde mappen of andere provisioningstaken worden uitgevoerd. Via SSH kunnen configuratiescripts of installatieopdrachten op afstand worden uitgevoerd. Shared folders kunnen daarnaast gebruikt worden om bestanden tussen host en VM uit te wisselen.

Linux provisioning

```
1 channel.exec(&format!("echo {} | sudo -S bash /media/sf_shared/{}",  
    password, scriptname))  
2 .expect("Command execution failed");
```

Codefragment 3.1: Uitvoering van het provisioningsscript in Linux

Windows provisioning

```
1 channel.exec(&format!(".\\psexec.exe -s -accepteula powershell.exe  
    -ExecutionPolicy Bypass -File \"Z:\\{}\"", scriptname))  
2 .expect("Command execution failed ");
```

Codefragment 3.2: Uitvoering van het provisioningsscript in Windows via psexec

4

Uitbreidingen

Naast het opstarten en provisionen voorziet de tool ook extra functies die het gebruik in labo-omgevingen verbeteren, zoals snapshots, foutafhandeling en logging. Deze functies maken het systeem robuuster en praktischer voor dagelijks gebruik. Ze zorgen er ook voor dat de gebruiker sneller kan herstellen wanneer er iets misloopt.

4.0.1. Snapshotting

Voor het provisionen kan een snapshot worden gemaakt, zodat de gebruiker bij fouten altijd kan terugkeren naar een stabiele toestand. Dit verhoogt de betrouwbaarheid, maar kost wel extra opslagruimte. In een onderwijscontext kan dit nuttig zijn om snel opnieuw te beginnen zonder alles opnieuw te moeten installeren.

4.0.2. Priority Queue

De VM's kunnen in een vaste volgorde worden opgestart via een prioriteitsmechanisme. Die volgorde wordt als extra parameter in het configuratiebestand opgenomen. Dit kan handig zijn wanneer bepaalde machines afhankelijk zijn van andere machines die eerst beschikbaar moeten zijn.

4.0.3. Kill

Deze functie zorgt ervoor dat VirtualBox geforceerd wordt afgesloten. Ze werd toegevoegd omdat VirtualBox in sommige gevallen niet correct reageert bij het beheren van virtuele machines, bijvoorbeeld wanneer het programma vastloopt of foutcodes blijft geven.

4.0.4. Ondersteuning

Ondersteunde besturingssystemen

De code past zich automatisch aan op basis van het besturingssysteem waarop de provisioner wordt uitgevoerd. Hierdoor kan de juiste gebruikersmap of tijdelijke opslaglocatie worden gebruikt, afhankelijk van of de tool op Windows of Linux draait.

```

1 pub fn get_home_dir() -> std::path::PathBuf {
2     std::env::var("HOME")
3     .map(std::path::PathBuf::from)
4     .or_else(|_| {
5         std::env::var("USERPROFILE")
6         .map(std::path::PathBuf::from)
7         .map_err(|_|
8             std::io::Error::new(std::io::ErrorKind::NotFound, "Home
9             directory not found"))
10    })
11    .expect("Failed to determine home directory")
12 }
```

Codefragment 4.1: Voorbeeld van hoe de doelmap dynamisch wordt opgebouwd op basis van het besturingssysteem.

```

1 if cfg!(target_os = "windows") {
2     //code fragment
3 }
```

Codefragment 4.2: Zoeken naar het os type

Ondersteunde besturingssystemen voor provisioning

Er werd gekozen voor ondersteuning van Linux en Windows, omdat dit de twee besturingssysteemtypes zijn die binnen HOGENT het vaakst gebruikt worden. Voor deze bachelorproef werden vier boxes voorzien.

AlmaLinux

AlmaLinux is de gekozen Linuxdistributie binnen HOGENT voor het vak Linux. Daarom wordt deze distributie ook ondersteund binnen de provisioner.

Ubuntu

Ubuntu is een van de meest gebruikte Linuxdistributies en werd daarom opgenomen als ondersteunde omgeving. [//https://www.openlogic.com/blog/top-enterprise-linux-distributions](https://www.openlogic.com/blog/top-enterprise-linux-distributions)

Windows 10

Windows 10 werd gekozen als clientbesturingssysteem voor de Windowsvakken.

Windows Server 2025

Windows Server 2025 werd gekozen als serverbesturingssysteem voor de Windowsvakken.

één voor AlmaLinux, één voor Ubuntu en twee voor Windows-omgevingen. De Windows boxes zijn voor Windows 10 en voor Windows Server 2025

4.0.5. Dependencies

Voor de implementatie worden onder andere de volgende Rust-crates gebruikt:

- `serde` voor het serialiseren en deserialiseren van gegevens. Deze crate werd gebruikt in combinatie met `toml` om configuratiebestanden vlot om te zetten naar Rust-structuren en omgekeerd.
- `ssh2` voor SSH-verbindingen. Deze crate werd gekozen voor cross-platform compatibiliteit, omdat de ingebouwde SSH-oplossingen van Linux en Windows te verschillend zijn en niet op dezelfde manier met authenticatie omgaan. De crate is een Rust-binding voor `libssh2`, een SSH-clientbibliotheek, en is dual-licensed onder Apache 2.0 en MIT. (**ssh2Crate**)
- `toml` voor het inlezen van configuratiebestanden. Deze crate werd gekozen omwille van de eenvoud en duidelijkheid van het importeren van data uit een TOML-bestand. TOML is een leesbaar configuratieformaat dat vlot kan worden omgezet naar Rust-structuren. De crate is een `serde`-compatibele TOML-decoder en -encoder voor Rust en is dual-licensed onder Apache 2.0 en MIT. (**tomlCrate**)
- `clap` voor command-line argumenten. Deze crate werd gekozen omdat ze een eenvoudige en gebruiksvriendelijke manier biedt om CLI-argumenten te verwerken. De crate is dual-licensed onder Apache 2.0 en MIT. (**clapCrate**)
- `indexmap` voor het bewaren van invoervolgorde bij configuratiegegevens. Deze crate werd gebruikt omdat de volgorde van de VM's belangrijk is bij het opstarten en provisionen, iets wat met een standaard `HashMap` niet gegarandeerd wordt.
- `crossterm` voor terminalinteractie en console-uitvoer. Deze crate werd gebruikt om de command-line ervaring overzichtelijker en gebruiksvriendelijker te maken.
- `PSTools` voor Windows-specifieke uitvoering van scripts als SYSTEM-gebruiker.

5

Benchmark

5.0.1. Snelheidsvergelijking met Vagrant

De uiteindelijke implementatie wordt vergeleken met Vagrant om na te gaan of de nieuwe provisioner sneller of trager is. Hiervoor wordt de provisioningtijd gemeten onder gelijkaardige omstandigheden. De vergelijking gebeurt op basis van een beperkt aantal testgevallen, namelijk drie Linux-VM's en twee Windows-VM's. Op die manier kan worden nagegaan of de gekozen aanpak een echte verbetering vormt ten opzichte van de bestaande oplossing.

Deze testen werden uitgevoerd op een computer met 32 GB DDR4-geheugen aan 3200 MHz, een AMD Ryzen 7 5800H-processor en een NVIDIA GeForce RTX 3060-laptopgpu met een vermogen van 90 W op Windows 11 25H2.

- **Eén VM:** Tijd vanaf het opstarten van de VM tot de start van het provisioning-proces. Deze meting wordt stopgezet op het moment dat de provisioning begint, omdat dit een beter beeld geeft van de snelheid van de provisioner zelf. Het uitvoeren van het provisioning-script wordt namelijk mede beïnvloed door externe factoren zoals internetverbindingen, wat de meting minder stabiel maakt.
- **Meerdere VM's:** Totale uitvoertijd van alle VM's, na elkaar uitgevoerd. Deze meting is realistischer omdat ook de uitvoering van de scripts wordt meegerekend, waardoor een meer volledige vergelijking met meerdere VM's mogelijk is.

Windows

Voor Windows worden een Windows Server 2025-omgeving en een Windows 10-clientomgeving getest. De Windows-gevallen zijn gebaseerd op de SEP-opdracht van vorig jaar en bevatten een domeincontroller en een client die via DHCP wordt toegevoegd aan het domein. Eerst wordt de tijd gemeten tot de provisioning van

één VM volledig afgerond is. Daarna wordt ook de totale tijd gemeten wanneer meerdere VM's worden opgestart en alle scripts volledig zijn uitgevoerd.

Test	1	2	3	Gemiddelde
Eén VM: Vagrant	x	x	x	x
Eén VM: RustProv	x	x	x	x
Meerdere VM's: Vagrant	x	x	x	x
Meerdere VM's: RustProv	x	x	x	x

Tabel 5.1: Vergelijking van de Windows-tijden tussen Vagrant en RustProv in seconden.

Linux

Voor Linux worden drie virtuele machines op een gelijkaardige manier geanalyseerd. Ook hier wordt gekeken naar de tijd tot de provisioning van één VM volledig afgerond is. Vervolgens wordt de totale uitvoeringstijd gemeten voor het scenario met meerdere VM's, waarbij alle scripts uitgevoerd en afgerond zijn-. Op die manier kan een eerlijke vergelijking gemaakt worden tussen Vagrant en RustProv.

Test	1	2	3	Gemiddelde
Eén VM: Vagrant	x	x	x	x
Eén VM: RustProv	x	x	x	x
Meerdere VM's: Vagrant	x	x	x	x
Meerdere VM's: RustProv	x	x	x	x

Tabel 5.2: Vergelijking van de Linux-tijden tussen Vagrant en RustProv in seconden.

5.1. Requirements-analyse

Dit hoofdstuk beschrijft de minimale vereisten waaraan RustProv moet voldoen om als volwaardige provisioner te kunnen functioneren binnen de context van deze bachelorproef. De requirements-analyse vertrekt vanuit de algemene betekenis van provisioning, maar wordt vervolgens toegespitst op de concrete noden van de opleiding en de beoogde testomgevingen. Het doel van deze analyse is om vast te leggen welke functionaliteiten absoluut noodzakelijk zijn en welke uitbreidingen de tool praktischer en robuuster maken.

5.2. Minimale vereisten

Om als volwaardige provisioner te kunnen functioneren, moet **RustProv** minstens een aantal basisfuncties ondersteunen. Deze functies vormen samen de minimale keten die nodig is om een bruikbare virtuele omgeving automatisch op te zetten.

1. **Configuratie kunnen inlezen**

Een eerste vereiste is dat de tool configuratiegegevens kan inlezen uit een extern bestand. Hierdoor worden instellingen niet hardgecodeerd in de broncode en kan het gedrag van de tool aangepast worden zonder de implementatie te wijzigen. In RustProv gebeurt dit via TOML-configuratiebestanden, wat de leesbaarheid en uitbreidbaarheid van de configuratie verhoogt.

2. **Een virtuele machine kunnen aanmaken of herkennen**

Een provisioner moet in staat zijn om een virtuele machine aan te maken wanneer die nog niet bestaat, of een bestaande machine te herkennen wanneer die reeds aanwezig is. Deze vereiste is noodzakelijk om herbruikbare workflows te ondersteunen en om te vermijden dat telkens een volledig nieuwe omgeving moet worden opgebouwd.

3. **Een VM kunnen starten en stoppen**

Naast het aanmaken van een VM moet de tool deze ook gecontroleerd kunnen starten en stoppen. Zonder deze mogelijkheid kan het provisioningproces niet op een betrouwbare manier worden uitgevoerd. Voor RustProv is dit extra belangrijk omdat meerdere scenario's gebruikmaken van zowel één enkele VM als van een opstelling met meerdere onderling afhankelijke VM's.

4. **Netwerktogang kunnen voorzien**

Een provisioner moet de gastmachine kunnen bereiken via een netwerkverbinding, typisch via SSH of een gelijkaardig communicatiekanaal. Zonder deze stap is het niet mogelijk om automatisch configuratieopdrachten uit te voeren op de VM. In RustProv wordt hiervoor gewerkt met SSH-gebaseerde toegang, waarbij netwerkregels tijdelijk worden ingesteld zodat de machine bereikbaar wordt tijdens de provisioningfase.

5. **Scripts of configuratiestappen kunnen uitvoeren**

Het eigenlijke provisionen bestaat uit het uitvoeren van scripts of andere configuratiestappen op de gastmachine. Dit is de kernfunctie van de tool: zonder deze mogelijkheid blijft enkel VM-beheer over. De provisioner moet dus niet alleen toegang krijgen tot de machine, maar ook effectief een script kunnen starten en opvolgen tot de configuratie voltooid is.

6. **Fouten kunnen signaleren en herstellen**

Een laatste minimale vereiste is foutafhandeling. Wanneer een stap in het proces mislukt, moet de gebruiker geïnformeerd worden en moet de tool het proces gecontroleerd kunnen afbreken of herstellen. Dit is noodzakelijk om te vermijden dat een onbruikbare of half geconfigureerde omgeving achterblijft. In een onderwijscontext is dit extra belangrijk, omdat instabiele omgevingen leiden tot tijdverlies en verwarring.

5.3. Vereisten voor RustProv

Op basis van deze algemene minimale vereisten worden voor RustProv meer concrete functionele eisen geformuleerd. Deze vereisten zijn afgeleid uit de doelstelling van de bachelorproef en uit de nood aan ondersteuning voor zowel eenvoudige als meer complexe VM-scenario's binnen de opleiding.

Ondersteuning voor Windows en Linux RustProv moet zowel Windows- als Linux-VM's kunnen ondersteunen. Die keuze is logisch, omdat beide platformen binnen de opleiding gebruikt worden en omdat de bachelorproef expliciet gericht is op provisioning voor beide types gastbesturingssystemen.

Ondersteuning voor één of meerdere VM's De tool moet inzetbaar zijn voor zowel een eenvoudige opstelling met één VM als voor uitgebreidere scenario's met meerdere VM's. Dat is nodig omdat de experimenten en benchmarks niet beperkt blijven tot één enkele machine, maar ook meerlagige opstellingen omvatten.

Externe configuratie via TOML Alle belangrijke instellingen moeten via een extern TOML-bestand kunnen worden beschreven. Op die manier wordt de opstelling declaratiever en eenvoudiger aanpasbaar. Deze vereiste ondersteunt ook de reproduceerbaarheid van experimenten, omdat dezelfde configuratie opnieuw gebruikt kan worden voor meerdere testruns.

SSH-gebaseerde uitvoering van scripts De uitvoering van provisioningsscripts moet via SSH of een vergelijkbare benadering gebeuren. RustProv gebruikt hiervoor een aparte SSH-library, omdat de ingebouwde oplossingen van Windows en Linux te veel van elkaar verschillen om op een uniforme manier te gebruiken. Deze keuze ondersteunt de cross-platform doelstelling van de tool.

VM-status kunnen opvragen De gebruiker moet de status van een VM kunnen opvragen. Deze functionaliteit is nodig om te bepalen of een machine reeds actief is, nog moet worden opgestart of correct werd afgesloten. Dit draagt bij tot een betrouwbaarder provisioningproces en voorkomt onnodige dubbele acties.

Snapshots kunnen maken en herstellen RustProv moet snapshots kunnen aanmaken en, waar nodig, gebruiken om sneller terug te keren naar een eerdere toestand. Deze vereiste is vooral relevant tijdens testen en herhaalde provisioningruns, omdat ze het mogelijk maakt om niet telkens een volledige VM opnieuw te moeten opbouwen.

VM's stoppen, geforceerd stoppen en verwijderen Naast gewone stopfunctionaliteit moet de tool ook ondersteuning bieden voor geforceerd stoppen en verwijderen van VM's. Dit is relevant omdat virtuele machines in de praktijk niet

altijd correct reageren op standaardcommando's. Door deze mogelijkheden te voorzien, blijft de tool ook bruikbaar in foutszenario's.

Tijdelijke netwerkregels kunnen opschonen Omdat tijdens de provisioning tijdelijk netwerkregels kunnen worden toegevoegd, moet RustProv deze regels ook opnieuw kunnen verwijderen wanneer ze niet langer nodig zijn. Dit voorkomt conflicten met andere toepassingen en helpt om de hostomgeving netjes achter te laten na afloop van het proces.

5.4. Functionele en niet-functionele vereisten

De requirements van RustProv kunnen samengevat worden als een combinatie van functionele en niet-functionele vereisten.

Functionele vereisten

- Inlezen van configuratie uit een extern bestand.
- Aanmaken, herkennen, starten en stoppen van VM's.
- Uitvoeren van provisioning scripts op Windows- en Linux-VM's.
- Ondersteuning voor één of meerdere VM's binnen één scenario.
- Opvragen van VM-status, maken van snapshots en verwijderen van VM's.

Niet-functionele vereisten

- Herhaalbaarheid van experimenten en configuraties.
- Bruikbaarheid binnen een onderwijs- en labo-omgeving.
- Ondersteuning voor verschillende besturingssystemen.
- Betrouwbare foutafhandeling en gecontroleerd opschonen van tijdelijke configuraties.
- Voldoende flexibiliteit om nieuwe scenario's later toe te voegen.

5.5. Afbakening

Deze bachelorproef richt zich op een proof-of-concept en niet op een volledig productierijp provisioningplatform. Dat betekent dat de focus ligt op de kernfunctionaliteit die nodig is om geautomatiseerde Windows- en Linuxomgevingen te kunnen opzetten en vergelijken met een bestaande aanpak zoals Vagrant. Vereisten zoals uitgebreide logging, een grafische gebruikersinterface of brede ondersteuning voor andere hypervisors vallen buiten de primaire scope van dit onderzoek, tenzij ze rechtstreeks bijdragen aan de evaluatie van RustProv binnen de gekozen experimenten.